

ScioTable

We're going to gradually build a table module from scratch.

In general:

- Do the parts in order. **The evaluation criteria is: 1) Progress on the parts, with correct output & satisfying the noted constraints 2) Reasonably clean code.**
- Unless noted otherwise, run time complexity does not need to be perfectly optimal, but try to avoid doing things super inefficiently.
- Feel free to add more public/private functions if useful
- Assume input is well-formed
- Feel free to use the internet for any coding-related questions
- No third party libraries but you can use standard libraries
- No need to write full-on unit tests but of course testing is encouraged

V0

We want to implement a table where the columns have names and a record is a given row in the table. The values are all integers. Initially, it should support the following operations:

```
class ScioTable:
    def __init__(self, column_names: string[]):
        """Constructor, creates a table with the given columns"""

    def insert_row(self, row: int[]):
        """Adds a row to the end of the table. Row will have the same length as the number of
        columns, and index i of the row corresponds to column i"""

    def read(self, columns: string[]):
        """Prints to the console the desired column headers in the order given followed by
        the values of the provided columns. If columns is an empty array, prints all columns.
        Different rows must be separated by a new line and rows must be printed in order of
        insertion."""
```

For the column names in `read()`, you can **assume that the columns exist**. You can also assume similarly for the rest of this problem.

V1

Support a slice function:

```
def slice(self, columns: string[]):  
    """Returns a new ScioTable instance which only has the passed in columns."""
```

You should now be able to do `t1.slice(["a"]).read([])`

V2

Support functions for adding/deleting a new column to the table.

```
def add_column(self, column: string):  
    """Returns a new ScioTable instance with the additional new column to the table. For  
    existing rows, values for this column should be set to 0. You can assume the column  
    name doesn't match any of the existing column names."""
```

```
def delete_column(self, column: string):  
    """Returns a new ScioTable instance which removes the given column. You can assume  
    the column always exists."""
```

V3

Please let your interviewer know when you've read this part.

```
def inner_join(self, table2: Table, column: string):  
    """Returns a new ScioTable instance with columns from this table and table2 for  
    whenever there are records with matching values for the given column. There is only  
    one shared column between the two tables, which is the given column. The ordering of  
    the columns in the new table should be {self.columns - column}{column}{table2.columns  
    - column}. See examples on the next page.
```

For inner_join() only:

- Order of the rows in the output table does not matter.
- The column values are **unique within each table** (i.e. there will only be at most one match for each row).
- The values of the table are not hashable - meaning that we cannot use the values as keys to a HashMap/dictionary. They are comparable.
- This method will be used infrequently
- **We want the runtime complexity for this method to be better than $O(M * N)$, where M, N are the number of rows in the tables and you can assume M, N are large and similar in magnitude.**

Example 1:

Current table:

A	B
---	---

1	3
---	---

2	6
---	---

Table2:

B	C
---	---

6	8
---	---

5	3
---	---

Then `table.inner_join(table2, "B")` should return a table as follows since B = 6 in both tables:

A	B	C
---	---	---

2	6	8
---	---	---

Example 2 (larger):

Current table:

A	B
---	---

1	3
---	---

2	6
---	---

7	9
---	---

4	4
---	---

Table2:

B	C
---	---

6	8
---	---

5	3
---	---

4	2
---	---

Then `table.inner_join(table2, "B")` should return a table as follows since B = 6 and B = 4 matches in both tables:

A	B	C
---	---	---

2	6	8
---	---	---

```
4 | 4 | 2
"""
```

V3 - continued

```
def outer_join(self, table2: Table, column: string):
    """Returns a new ScioTable instance with columns from this table and table2 for
    whenever there are records with matching values for a column, and for values that
    don't match, inserts a record with only values for the columns in the table it came
    from and 0 for all columns in the other table. There is only one shared column
    between the two tables, which is the given column. The ordering of the columns in the
    new table should be {self.columns - column}{column}{table2.columns - column}.
```

Order of the rows does not matter. Assume the input column exists in both tables. **The constraints from the inner_join() part do not apply!**

Example 1:

Current table:

```
A | B
-----
1 | 3
2 | 6
```

Table2:

```
B | C
-----
6 | 8
5 | 3
```

Then table.outer_join(table2, "B") should return a table as follows:

```
A | B | C
-----
1 | 3 | 0
2 | 6 | 8
0 | 5 | 3
```

Example 2:

Current table:

```
A | B
-----
1 | 3
2 | 6
```

```
7 | 9
4 | 4
```

Table2:

```
B | C
```

```
-----
6 | 8
5 | 3
4 | 2
```

Then `table.outer_join(table2, "B")` should return a table as follows:

```
A | B | C
```

```
-----
1 | 3 | 0
2 | 6 | 8
7 | 9 | 0
4 | 4 | 2
0 | 5 | 3
"""
```

V4

Suppose `insert_row()` successfully inserts a row 60% of the time and does nothing 40% of the time due to write failures. Add the following implemented function to `ScioTable`:

Python

```
import random
def insert_row_noisy(self, row):
    if random.random() < 0.4:
        return
    self.insert_row(row)
```

Java

```
import java.lang.Math;
public void insert_row_noisy(List<int> row) {
    if (Math.random() < 0.4) return;
    insert_row(row);
}
```

We want to build the RobustScioTable, which has the same functionality and requirements as ScioTable V0, but insertions succeed at least 90% of the time, using multiple ScioTables given a ScioTable's insert_row_noisy() succeeds 60% of the time:

```
class RobustScioTable:
    def __init__(self, column_names: string[]):
        """Constructor, creates a table with the given columns"""

    def insert_row(self, row: int[]):
        """Adds a row to the end of the table. Row will have the same length as the number of
        columns, and index i of the row corresponds to column i"""

    def read(self, columns: string[]):
        """Prints to the console the desired column headers in the order given followed by
        the values of the provided columns. If columns is an empty array, prints all columns.
        Different rows must be separated by a new line and rows must be printed in order of
        insertion."""
```

V5

Support export & import functions (outside of the class)

```
def export_table(filename: string, table: ScioTable, columns: string[]):
    """Write the table (only the selected columns) as a csv to the given filename."""

def import_table(filename: string, columns: string[]):
    """Reads the selected columns from csv and returns a ScioTable."""
```